

file Handling in C

In an program we input data through the keyboard and we get output on the screen.

After termination of program, all the entered data is lost because primary memory is volatile.

If we want to use the data later then it becomes necessary to keep it in permanent storage device.

So we use file to save the data in permanent storage.

File is a sequence of bytes / memory or block where we can store data permanently.

Types of files in C :

A file can be classified into two types based on the way the file stores the data. They are as follows:

Text files

Binary files

Text files :

A text file contains data in the form of ASCII characters and is generally used to store a stream of characters.

Each line in a text file ends with a new line character (\n).

It can be read or written by any text editor.

They are generally stored with .txt file extension.

Text files can also be used to store the source code.

Binary files :-

A binary file contains data in binary form instead of ASCII characters. They contain data.

The binary files can be created only from within a program and their contents can only be read by a program.

More secure as they are not easily readable.

They are generally stored with .bin file extension.

To handle the file we need pointer.

A pointer provides a way to access memory location directly.

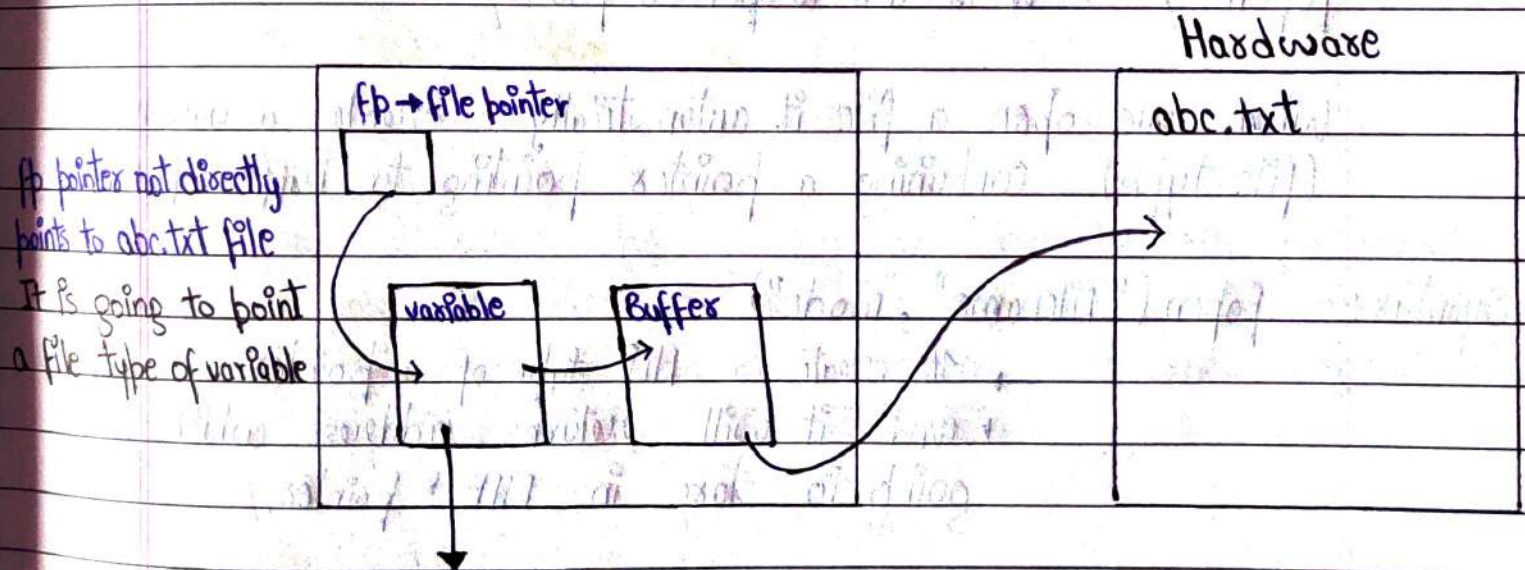
We need a FILE pointer.

A file pointer is used in file handling to perform all file operations.

FILE pointer

Syntax: `FILE *fp;` This pointer contains address of file type of variable.

If we go more deep



Here we have a pointer that points to buffer file.

Buffer is an area in memory where the data is temporarily stored before being written to the file.

When we open a file a buffer automatically associated with its file pointer. Whatever data we send to the

File is not immediately written to the file. First it is sent to the buffer and when the buffer is full, its contents are written to the file.

When the file is closed, all the contents of buffer are automatically written to the file even if the buffer is not full.

This is called flushing the buffer. we can also explicitly flush the buffer by a function `fflush()`.

- `fopen()` → it is use to open a file

When we open a file it automatically creates a variable (file type) containing a pointer pointing to buffer file.

Syntax: `fopen("filename", "mode")`

- It creates a FILE type of a pointer
- and it will returns address which is going to store in FILE * pointer.

Types of mode are :

1. "w" (write)

If the file doesn't exist then this mode creates a new file for writing, and if the file already exists then the previous data is erased and the new data entered is written to the file.

2. "a" (append)

If the file doesn't exist then this mode creates a new file, and if the file already exists then the new data entered is appended at the end of existing data. In this mode, the data existing in the file is not erased as in "w" mode.

3. "r" (read)

This mode is used for opening an existing file for reading purpose only. The file to be opened must exist and the previous data of file is not erased. ~~new data and modify existing data~~

4. "w+" (write + read)

This mode is same as "w" mode but in this mode we can also read and modify the data. If the file doesn't exist then a new file is created and if the file exists then previous data is erased.

5. "r+" (read + write)

This mode is same as "r" mode but in this mode we can also write and modify existing data. The file to be opened must exist and the previous data of file is not erased.

6. "a+" (append + read)

This mode is same as "a" mode but in this mode we can also ~~we~~ read the data stored in the file. If the file doesn't exist, a new file is created and if the file already exists then new data is appended at the end of existing data. We cannot modify existing data in this mode.

Structure of a file program :

```
#include <stdio.h>
int main ( )
{
    FILE *fp;
    fp = fopen("filename", "mode");
    -----
    -----
    fclose(fp);
    return 0;
}
```


1. Writing in a file

```
#include <stdio.h>
```

```
int main ()
```

```
{
```

```
FILE *fp;
```

```
char ch = 'a';
```

```
fp = fopen("abc.txt", "w");
```

```
fputc(ch, fp);
```

```
fclose(fp);
```

```
}
```

Built-function use for it ÷

- fputc - To write a character in a file.

Syntax ÷ fputc(variable, file pointer)

↓
In which character is store

If you want to take input from user then ÷

```
#include <stdio.h>
```

```
int main ()
```

```
{
```

```
FILE *fp;
```

```
char ch;
```

```
fp = fopen("abc.txt", "w");
```

```
printf("Enter a character = ");
```

```
scanf("%c", &ch);
```

```
fputc(ch, fp);
```

```
}
```



```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
FILE *fp;
```

```
fp = fopen("abc.txt", "w");
```

```
fputs("Grauxav Kondpal", fp);
```

```
}
```

• fputs - To write a string in a file
Syntax : fputs (variable, file pointer)

↓
In which string is stored

We can also input on string using fputc :

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
FILE *fp;
```

```
int i;
```

```
char str[] = "Grauxav Kondpal";
```

```
fp = fopen("abc.txt", "w");
```

```
for(i=0; i!=strlen(str); i++)
```

```
{
```

```
fputc(str[i], fp);
```

```
}
```

```
fclose(fp);
```

```
}
```



```
#include <stdio.h>
int main ()
{
```

- **fprintf** ÷ Formatted output function
÷ It is use to write mixed datatype in a file

```
FILE *fp ;
int a = 30;
char str[] = "Gaurav Kondpal";
fp = fopen("abc.txt", "w");
fprintf(fp, "%d %s", a, str);
fclose(fp);
}
```

syntax ÷ fprintf (file pointer, format specifier, variable)

Sometime fopen is unable to open an file due to some error and it not return the address of a pointer of type FILE instead ~~it~~ it return NULL.

So it always better to check it.

```
#include <stdio.h>
#include <stdlib.h>
{
```

```
FILE *fp ;
int a = 30;
char str[] = "Gaurav Kondpal";
fp = fopen("abc.txt", "w");
if (fp == NULL)
{
    printf("File is not created/opened");
    exist(1);
}
fprintf(fp, "%d %s", a, str);
fclose(fp);
}
```


Date _____
Page _____

End of file :

The file reading functions need to know the end of file so that they can stop reading.

When the end of file is reached, the operating system sends an end of file signal to the program.

When the program receives this signal, the file reading functions return EOF, which is a constant defined in the file `stdio.h` and its value is -1.

EOF is an integer value, so make sure that the return value of the function is assigned to an integer variable.

`fgetc()`, `fprintf()` return EOF and `fgets()` returns NULL on end of file. But these functions return the same value both on end of file and on error.

So if a function stops reading we can't make out whether the end of file was reached or an error was encountered.

We can check separately for these two conditions with the help of :

`ferror()`

`feof()`

- **feof()**

feof() is used to check the end of file condition. It returns a nonzero value if end of file has been reached otherwise it returns zero.

- **ferror()**

ferror() is used for detecting any error occurred during read or write operations on a file. It returns a nonzero value if an error occurs, i.e. if the error indication for that file is set, otherwise it returns zero.

Read a file ()

Built in function use for it

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main ( )
```

```
{
```

```
FILE * fp;
```

```
char ch;
```

```
fp = fopen("abc.txt", "r");
```

```
if ( fp == NULL)
```

```
{
```

```
    printf("unable to open the file");
```

```
    exit(1);
```

```
}
```

```
ch = fgetc(fp);
```

```
while (ch != EOF)
```

```
{
```

```
    printf("%c", ch);
```

```
    ch = fgetc(fp);
```

```
}
```

```
fclose(fp)
```

```
}
```

- fgetc : To read a single character from a file at a time

syntax : fgetc (file pointer)

```
while (!feof(fp))
```

```
{
```

```
    ch = fgetc(fp);
```

```
    printf("%c", ch);
```

```
}
```

```
fclose(fp)
```

```
}
```

check end of the file
is reached or not


```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main()
```

```
{
```

```
FILE *fp;
```

```
char ch;
```

```
fp = fopen("abc.txt", "r");
```

```
if (fp == NULL)
```

```
{
```

```
printf("unable to open the file");
```

```
exit(1);
```

```
}
```

```
while((ch = fgetc(fp)) != EOF)
```

```
printf("%c", ch);
```

```
fclose(fp);
```

```
return 0;
```

```
}
```


- `fgets` : To read whole string from a file.

syntax : `fgets(variable, n, file pointer)`

- `fgets` reads at most $n-1$ characters from the file

↓ ↓
In which string no. of character
is store we want to
read

- The maximum value of n can be equal to size of the string. But it reads at most $n-1$ character and one is left for null character.

- `fgets()` stop reading after $n-1$ character
- When it reach to the end of first line it stop reading
- Once it find new line character it stops reading.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main ( )
```

```
{
```

```
FILE *fp;
```

```
char str[20];
```

```
fp = fopen("abc.txt", "r");
```

```
if ( fp == NULL)
```

```
{
```

```
printf("unable to open the file");
```

```
exit(1);
```

```
}
```

```
fgets(str, 10, fp);
```

```
printf("%s", str);
```

```
fclose(fp)
```

```
}
```


If you want to print all lines of an file so it better to use an loop.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main()
```

```
{
```

```
FILE *fp;
```

```
char str[50];
```

```
fp = fopen("abc.txt", "r");
```

```
if (fp == NULL)
```

```
{
```

```
printf("unable to open the file");
```

```
exit(1);
```

```
}
```

```
while (!feof(fp))
```

// it will print all lines of a file with new line character.

```
fgets(str, 45, fp);
```

```
printf("%s", str);
```

```
}
```

```
fclose(fp);
```

```
}
```


- fscanf() ÷ To read set of characters from file.

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
FILE *fp;
```

```
int a;
```

```
char str[100];
```

```
fp = fopen("abc.txt", "r");
```

```
if (fp == NULL);
```

```
{
```

```
printf("error");
```

```
exit(1);
```

```
}
```

```
fscanf(fp, "%s %d", str, &a);
```

```
printf("%s %d", str, a);
```

```
fclose(fp);
```

```
}
```

syntax ÷ fscanf (file pointer, format specifier, variable)

Append Mode in file handling :

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    FILE * fp;
    char str[50];
    fp = fopen("abc.txt", "a");
    if (fp == NULL)
    {
        printf("unable to open a file");
        exit(1);
    }
    printf("Enter the content you want to append");
    gets(str);
    fprintf(fp, "%s", str);
    printf("Successfully appended");
    fclose(fp);
}
```


x⁺ mode

Reading / Writing

- You can also modify the file from any line or any specific point.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main()
```

```
{
```

```
FILE *fp;
```

```
char a;
```

```
fp = fopen("abc.txt", "x+");
```

```
if (fp == NULL)
```

```
{
```

```
printf("unable to open the file");
```

```
exit(1);
```

```
}
```

```
a = fgetc(fp);
```

```
printf("%c", a); // I
```

```
fputs("Jenny", fp);
```

```
fclose(fp);
```

```
}
```

abc.txt

I am Gaurav
Jenny

By default it write from starting

w+ mode

reading / Writing

Create new file (if file doesn't exist)

file exist then / erase the previous content

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main()
```

```
{
```

```
FILE * fp;
```

```
char ch;
```

```
fp = fopen("abc.txt", "w+");
```

```
if (fp == NULL) {
```

```
printf("unable to open the file");
```

```
exit(1);
```

```
}
```

```
fputs("Kandpal", fp);
```

```
while (!feof(fp))
```

```
ch = fgetc(fp);
```

```
printf("%c", ch);
```

```
fclose(fp);
```

```
}
```

It will not run

because at first only

we will get end of file

because cursor is after

Kandpal.

abc.txt

Gaurav Kandpal

Kandpal

→ now cursor
is here

It gonna to erase this & write

Date _____
Page _____

rewind()

Declaration: `rewind (file * pointer)`

This function is used to move the file position indicator to the beginning of the file.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main()
```

```
{
```

```
FILE * fp;
```

```
char ch;
```

```
fp = fopen("abc.txt", "w+");
```

```
if (fp == NULL)
```

```
{
```

```
printf("unable to open the file");
```

```
exit(1);
```

```
}
```

```
fputs("Kandpal", fp);
```

```
rewind(fp);
```

```
while (!feof(fp))
```

```
{
```

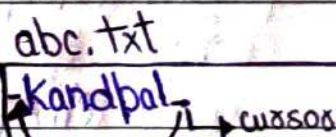
```
ch = fgetc(fp);
```

```
printf("%c", ch); // Kandpal
```

```
}
```

```
fclose(fp);
```

```
}
```



a⁺ mode

reading / appending

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main()
```

```
{
```

```
FILE *fp;
```

```
char ch;
```

```
fp = fopen("abc.txt", "a+");
```

```
if (fp == NULL)
```

```
{
```

```
printf("file doesn't exist");
```

```
exit(1);
```

```
}
```

```
fputs("khatrai", fp);
```

```
rewind(fp);
```

```
while (!feof(fp))
```

```
{
```

```
ch = fgetc(fp);
```

```
printf("%c", ch); // Jennykhatrai
```

```
}
```

```
fclose(fp);
```

```
}
```

abc.txt

Jennykhatrai

→

fseek()

It is used to move or change the position of the file pointer to a specified offset.

syntax :-

`fseek ( file pointer , offset , position )`



How many bits you want
to move the cursor

forward → +ve
backward → -ve

Datatype of :-

FILE
↑

`fseek ( file pointer , offset , position )`



return type (int)



long, int



int

position

SEEK_SET	0	Beginning of file
SEEK_CURRENT	1	Current position
SEEK_END	2	End of file


```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    FILE *fp;
    char ch;
    char str[50];
    fp = fopen("abc.txt", "r");
    if (fp == NULL);
    {
        printf("unable to open the file");
        exit(1);
    }

    fseek(fp, 6, SEEK_SET);
    ch = fgetc(fp);
    printf("%c", ch); // a

    fseek(fp, -3, SEEK_CUR); // m
    ch = fgetc(fp);
    printf("%c", ch);

    fseek(fp, -5, SEEK_END); // a
    ch = fgetc(fp);
    printf("%c", ch);

    fclose(fp);
}
```

abc.txt

I am Gaurav Kandpal

ftell()

This function returns the current position of the file position indicator.

It tells the position (index) of the file pointer with respect to the starting.

- It is also use for finding the length of the file.


```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main()
```

```
{
```

```
    char str[50];
```

```
    char ch;
```

```
    fp = fopen("abc.txt", "r");
```

```
    if (fp == NULL)
```

```
    {
```

```
        printf("unable to open the file");
```

```
        exit(1);
```

```
    }
```

```
    int pos = ftell(fp);
```

```
    printf("%d", pos); // 0 because we have open file in r mode so the cursor is present at starting
```

```
    fseek(fp, 6, SEEK_SET);
```

```
    printf("%d", ftell(fp)); // 6
```

```
    ch = fgetc(fp); → it will read/take space
```

```
    printf("%c", ch); → it will print space so the position of cursor
```

```
    printf("\n%d", ftell(fp)); will move one forward
```

```
    → 117
```

```
    fscanf(fp, "%s", str);
```

```
    printf("%s", str); // Kandpal
```

```
    printf("\n%d", ftell(fp)); // 14
```

abc.txt

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Gaurav Kandpal															20	

It is also use to find the length of the file.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main()
```

```
{
```

```
    FILE *fp;
```

```
    int length = 0;
```

```
    fp = fopen("abc.txt", "r");
```

```
    if (fp == NULL)
```

```
    {
```

```
        printf("unable to open the file");
```

```
        exit(1);
```

```
    }
```

```
    fseek(fp, 0, SEEK_END);
```

```
    length = ftell(fp);
```

```
    printf("length of the file = %d", length);
```

```
    fclose(fp);
```

```
}
```